



MCS-011: Frequently Asked Questions with Full Solutions

1. Algorithm and Flowchart Questions

(a) Print first n even natural numbers and their sum

Algorithm:

1. Start
2. Input n
3. Initialize $i = 1$, $\text{count} = 0$, $\text{sum} = 0$
4. Repeat while $\text{count} < n$:
5. If $i \% 2 == 0$:
 - Print i
 - $\text{sum} = \text{sum} + i$
 - $\text{count} = \text{count} + 1$
6. $i = i + 1$
7. Print sum
8. End

Flowchart: Refer to standard flowchart shapes (Start → Input → Loop → Condition → Output).

(b) Prime Number Check

Algorithm:

1. Start
 2. Input num
 3. If $\text{num} \leq 1$: Print "Not Prime" and End
 4. For $i = 2$ to $\text{sqrt}(\text{num})$:
 5. If $\text{num} \% i == 0$: Print "Not Prime" and End
 6. Print "Prime"
 7. End
-

(c) Factorial of a number

Algorithm:

1. Start
2. Input n
3. Set $\text{fact} = 1$
4. For $i = 1$ to n:
5. $\text{fact} = \text{fact} * i$
6. Print fact
7. End

2. Recursion-Based Questions

(a) Fibonacci Series

```
#include <stdio.h>
int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n-1) + fib(n-2);
}
int main() {
    int n = 10;
    for(int i = 0; i < n; i++)
        printf("%d ", fib(i));
    return 0;
}
```

(b) Factorial using Recursion

```
#include <stdio.h>
int factorial(int n) {
    if(n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);
}
int main() {
    int num = 5;
    printf("Factorial: %d", factorial(num));
    return 0;
}
```

3. Structure-Based Questions

(a) Student Record using Structure

```
#include <stdio.h>
struct Student {
    int roll;
    char name[50];
    float marks1, marks2, marks3, marks4;
    char grade;
};
int main() {
    struct Student s[10];
}
```

```

    for(int i = 0; i < 10; i++) {
        printf("Enter roll, name, 4 marks and grade: ");
        scanf("%d %s %f %f %f %c", &s[i].roll, s[i].name,
            &s[i].marks1, &s[i].marks2, &s[i].marks3, &s[i].marks4,
            &s[i].grade);
    }
    int roll_no;
    printf("Enter roll number to search: ");
    scanf("%d", &roll_no);
    for(int i = 0; i < 10; i++) {
        if(s[i].roll == roll_no) {
            printf("Marks: %.2f %.2f %.2f %.2f\n", s[i].marks1, s[i].marks2,
            s[i].marks3, s[i].marks4);
        }
    }
    return 0;
}

```

4. File Handling

(a) Copy one file to another

```

#include <stdio.h>
int main() {
    FILE *f1, *f2;
    char ch;
    f1 = fopen("input.txt", "r");
    f2 = fopen("output.txt", "w");
    if(f1 == NULL || f2 == NULL) {
        printf("File cannot be opened\n");
        return 1;
    }
    while((ch = fgetc(f1)) != EOF) {
        fputc(ch, f2);
    }
    fclose(f1);
    fclose(f2);
    return 0;
}

```

(b) Count lines in a file

```

#include <stdio.h>
int main() {
    FILE *f = fopen("input.txt", "r");
    char ch;
    int lines = 0;

```

```

    if(f == NULL) {
        printf("File error");
        return 1;
    }
    while((ch = fgetc(f)) != EOF) {
        if(ch == '\n') lines++;
    }
    fclose(f);
    printf("Total lines: %d", lines);
    return 0;
}

```

5. String Manipulation Without Built-in Functions

(a) Reverse a string

```

#include <stdio.h>
int main() {
    char str[100];
    int len = 0;
    printf("Enter string: ");
    gets(str);
    while(str[len] != '\0') len++;
    for(int i = len - 1; i >= 0; i--)
        printf("%c", str[i]);
    return 0;
}

```

(b) Copy string

```

#include <stdio.h>
int main() {
    char str1[100], str2[100];
    int i = 0;
    printf("Enter string: ");
    gets(str1);
    while(str1[i] != '\0') {
        str2[i] = str1[i];
        i++;
    }
    str2[i] = '\0';
    printf("Copied String: %s", str2);
    return 0;
}

```

6. Matrix Operation

```
#include <stdio.h>
int main() {
    int A[3][3], B[3][3], C[3][3], D[3][3];
    printf("Enter elements of A and B and C:\n");
    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++) scanf("%d", &A[i][j]);
    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++) scanf("%d", &B[i][j]);
    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++) scanf("%d", &C[i][j]);

    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++) {
            D[i][j] = 0;
            for(int k=0;k<3;k++)
                D[i][j] += A[i][k] * B[k][j];
            D[i][j] += C[i][j];
        }

    printf("Matrix D = A*B + C:\n");
    for(int i=0;i<3;i++) {
        for(int j=0;j<3;j++)
            printf("%d ", D[i][j]);
        printf("\n");
    }
    return 0;
}
```

7. Income Tax Calculation

```
#include <stdio.h>
int main() {
    float salary, tax = 0;
    printf("Enter salary: ");
    scanf("%f", &salary);
    if (salary < 500000)
        tax = 0;
    else if (salary < 800000)
        tax = (salary - 500000) * 0.1;
    else if (salary < 1000000)
        tax = 30000 + (salary - 800000) * 0.25;
    else
        tax = 80000 + (salary - 1000000) * 0.3;
    printf("Tax = %.2f\n", tax);
}
```

```
    return 0;
}
```

8. Call by Reference Example

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

9. Important Definitions

- **Structure:** A user-defined data type in C that allows grouping variables of different types.
- **Recursion:** A function calling itself to solve smaller instances of a problem.
- **Pointer:** A variable that stores the memory address of another variable.
- **Macro:** A fragment of code which has been given a name. Macros are defined using #define.
- **Algorithm:** A step-by-step procedure to solve a problem.
- **Flowchart:** A visual representation of the sequence of steps in a program.
- **File Handling:** Allows programs to read/write data to files using functions like fopen, fclose, fgetc, etc.
- **Call by Reference:** A method where actual memory address is passed, so changes affect original variables.